AlgebraKing: Um Sistema de Álgebra Computacional para o Terminal e para o Claude.

Guia do Usuário, Versão 0.0.1

Universidade Federal de Goiás

LEEDMOL - Laboratório de Estrutura Eletrônica e Dinâmica Molecular.



https://github.com/phfmatias/AlgebraKing

Sumário

1 O que é **1**

2 Instalação **1**

2.1 Requisitos . . . 1
2.2 Instalação automática . . . 1
2.3 Instalação manual . . . 2
2.4 Atualizar . . . 2
2.5 Desinstalar . . . 2

3 Primeiros passos **2**

3.1 As quatro formas de usar . . . 2
3.2 O REPL . . . 3
3.3 `;` mostra, `:` silencia . . . 3
3.4 `%`, `%%`, `%%%` . . . 4
3.5 2-D ou 1-D . . . 4
3.6 Scripts `.mpl` . . . 4
3.7 Código de saída . . . 4
3.8 Exemplos prontos . . . 4

4 Linguagem **5**

4.1 Atribuição, desatribuição, avaliação . . . 5
4.2 Procedimentos e funções-seta . . . 5
4.3 Derivada de operador `D`, composição `@` e `@@` . . . 6
4.4 Controle de fluxo . . . 6
4.5 `seq`, `add`, `mul`, `map`, `select`, `zip`, listas . . . 6
4.6 Matrix e Vector com colchetes angulares . . . 7
4.7 Pacotes: `with`, `:-` e `[]` . . . 8
4.8 `read` . . . 8
4.9 `try` / `catch` . . . 8
4.10 `assume`, `additionally`, `assuming`, `about` . . . 9
4.11 `Digits` e `evalf[n]` . . . 9
4.12 `printf`, `sprintf`, `print`, `lprint`, `latex` . . . 9
4.13 Precedência e operadores . . . 10

5 Matemática **10**

5.1 Álgebra e simplificação . . . 10
5.2 Polinômios . . . 11
5.3 Teoria dos números e funções inteiras . . . 11
5.4 Cálculo . . . 12

5.5 Resolução de equações 13
5.6 Álgebra linear (`LinearAlgebra`) 13
5.7 Transformadas (`inttrans`) 14
5.8 Cálculo vetorial (`VectorCalculus`) 15
5.9 Funções elementares e especiais 15
5.10 Diversos: tipos e estrutura 16

6 Usando com o Claude (MCP) 16
6.1 Registro automático 16
6.2 Registro manual 16
6.3 Claude Desktop 17
6.4 As 8 ferramentas 17
6.5 Como o veredito funciona 22
6.6 Sessões 23
6.7 Variáveis de ambiente 23

7 Limitações conhecidas 24

8 Solução de problemas 25

9 Validação 26

Guia completo do AlgebraKing. Todos os exemplos abaixo foram executados no binário `algebraking` (versão 0.0.1) e a saída mostrada é a saída real, em formato 1-D (o formato de scripts, de `-e` quando a saída não é um terminal e de `interface(prettyprint=0)`; veja "Primeiros passos").

1 O que é

AlgebraKing é um sistema de álgebra computacional para o terminal, inspirado na linguagem Maple. Foi escrito do zero, em **clean room** (nenhum código, binário ou documentação do Maple foi usado), e reproduziu 257/257 veredictos de uma bateria de verificação real, originalmente executada no Maple 18. A linguagem, os nomes das funções, a ordem dos argumentos e as convenções de retorno são os do AlgebraKing, descritos neste guia.

A arquitetura tem duas metades:

- **Rust** cuida da *linguagem*: lexer, parser, avaliação (variáveis, procedimentos, laços, `%`, pacotes, `with`), impressão no formato 1-D e o servidor MCP.
- **Python + SymPy** cuida da *matemática*: cada função matemática (`int`, `solve`, `dsolve`, `Eigenvalues`, …) é uma chamada a um motor (`engine/engine.py`) embutido no binário, que traduz as funções do AlgebraKing para o SymPy e devolve o resultado na convenção do AlgebraKing.

Por que existe? O uso principal é *conferir contas*: o Claude pega um passo da sua dedução (derivada, integral, solução de EDO, identidade) e pergunta ao AlgebraKing, por MCP, se está certo, recebendo `equal`, `not_equal` (com contraexemplo) ou `undetermined`. O segundo uso é um REPL de terminal. Quando o programa não sabe fazer algo, ele devolve a expressão sem avaliar ou dá erro; a regra é nunca devolver uma resposta errada com confiança.

Fora do escopo: interface gráfica, worksheets, janela de gráficos (`plot`) e geração de código (veja a seção 7).

---

2 Instalação

2.1 Requisitos

Item	Versão	Observação
Linux (testado em Ubuntu)	qualquer recente	
Python 3	≥ 3.8	o motor roda nele
`sympy` e `mpmath`	SymPy 1.14 (testado)	`pip install --user sympy mpmath`
Rust / `cargo`	estável recente	o `install.sh` instala via rustup se faltar
`claude` (Claude Code)	opcional	só para registrar o servidor MCP

2.2 Instalação automática

```
git clone https://github.com/phfmatias/AlgebraKing.git
cd AlgebraKing
./install.sh
```

O script é idempotente (pode rodar de novo sem problema) e faz, nesta ordem:

1. confere `python3 >= 3.8` e `import sympy, mpmath`; se faltar, roda `python3 -m pip install --user sympy mpmath` (se o pip recusar por "externally managed environment", repete com `--break-system-packages`, só no diretório do usuário);
2. confere `cargo`; se faltar, instala o rustup (perfil mínimo, sem perguntas) em `~/.cargo`;
3. `cargo build --release`;
4. cria o link simbólico `~/.local/bin/algebraking` apontando para `target/release/algebraking` (avisa, com a linha exata para o `~/.bashrc`, se `~/.local/bin` não estiver no `PATH`);
5. faz um autoteste: `algebraking -e 'int(x^2,x);'` precisa imprimir `x^3/3`;
6. registra o servidor MCP no Claude Code, se o comando `claude` existir (e pula se `claude mcp get algebraking` já responder);
7. imprime um resumo com os próximos passos.

Flag	Efeito
`--no-mcp`	não registra o servidor MCP no Claude Code
`--no-python-deps`	não confere nem instala `sympy`/`mpmath`
`--prefix DIR`	diretório do link simbólico (padrão: `~/.local/bin`)
`-h`, `--help`	ajuda

2.3 Instalação manual

```
python3 -m pip install --user sympy mpmath
cargo build --release
mkdir -p ~/.local/bin
ln -sfn "$PWD/target/release/algebraking" ~/.local/bin/algebraking
export PATH="$HOME/.local/bin:$PATH"          # coloque no ~/.bashrc
algebraking -e 'int(x^2,x);'                   # x^3/3
```

2.4 Atualizar

```
git pull && ./install.sh
```

2.5 Desinstalar

```
./uninstall.sh
```

Remove o link simbólico, o registro MCP (`claude mcp remove algebraking -s user`) e o cache do motor (`~/.cache/algebraking`). **Nunca** apaga a pasta do repositório; apague-a à mão se quiser. O histórico do REPL (`~/.algebraking_history`) também é mantido.

---

3 Primeiros passos

3.1 As quatro formas de usar

```
algebraking                    sessão interativa (REPL)
algebraking arquivo.mpl        executa um script
algebraking -e "código"        avalia uma string
algebraking mcp                servidor MCP no stdin/stdout
```

Também: `algebraking -V` (versão) e `algebraking -h` (ajuda).

3.2 O REPL

O prompt é >. Uma instrução pode ocupar várias linhas: enquanto estiver incompleta, o prompt vira . `Ctrl-C` descarta a entrada pendente, `Ctrl-D` (ou `quit;`) sai. O histórico (setas, `Ctrl-R`) é gravado em `~/.algebraking_history`.

```
$ algebraking
AlgebraKing 0.0.1 - symbolic math (SymPy engine). Type quit; to leave.
> f := x -> x^2*exp(-x):
> int(f(x), x=0..infinity);
2
> diff(f(x), x);
   2  -x        -x
- x ·e   + 2·x·e
> A := <<1|2>, <3|4>>:
> with(LinearAlgebra):
> MatrixInverse(A);
⎡-2    1  ⎤
⎢          ⎥
⎣3/2  -1/2⎦
> sqrt(2)/(1+x)^2;
   √2
────────
       2
(x + 1)
> %;
   √2
────────
       2
(x + 1)
> interface(prettyprint=0):
> diff(f(x), x);
-x^2*exp(-x)+2*x*exp(-x)
```

3.3 ; mostra, : silencia

`;` mostra o resultado e `:` o suprime (a última instrução pode ficar sem terminador):

```
> a := 5; b := a^2: c := a + b;
a := 5
c := 30
```

(Saída real de `algebraking -e 'a := 5; b := a^2: c := a + b;'`.)

3.4 `%`, `%%`, `%%%`

`%` é o último resultado mostrado, `%%` o penúltimo, `%%%` o antepenúltimo:

```
x^2 + y; %; %%;
x^2+y
x^2+y
x^2+y
```

3.5 2-D ou 1-D

No REPL, e em scripts/`-e` quando a saída **é um terminal**, os resultados saem em 2-D (como acima). Quando a saída vai para um pipe ou arquivo, saem em 1-D. Para forçar: `interface(prettyprint=0)` (1-D) ou `interface(prettyprint=1)` (2-D).

3.6 Scripts `.mpl`

Um script é um arquivo de texto com instruções da linguagem. O programa imprime os resultados das instruções terminadas em `;` e **para no primeiro erro, com código de saída 1**.

```
$ cat stop.mpl
a := 2;
b := a^10;
error "parou aqui";
c := 3;
$ algebraking stop.mpl; echo "rc=$?"
a := 2
b := 1024
Error, parou aqui
rc=1
```

3.7 Código de saída

Código	Significado
`0`	tudo certo (ou `quit` chamado no script)
`1`	erro de avaliação ou de sintaxe em script/`-e`; arquivo ilegível; motor não inicia
`2`	uso incorreto da linha de comando (opção desconhecida)

3.8 Exemplos prontos

A pasta `examples/` tem `basics.mpl`, `calculus.mpl`, `linear_algebra.mpl`, `ode.mpl` e `procedures.mpl`:

```
$ algebraking examples/calculus.mpl
f := x -> x^2*exp(-x)
x -> -x^2*exp(-x)+2*x*exp(-x)
exp(-1)
-sin(x)^2+cos(x)^2
```

```
1
arctan(x)
exp(1)
1+x+x^2/2+x^3/6+x^4/24+O(x^5)
Pi^2/6
x-x^2/2+x^3/3+O(x^4)
```

---

4 Linguagem

Todos os exemplos desta seção: `algebraking -e '...'`, saída em 1-D.

4.1 Atribuição, desatribuição, avaliação

Entrada	Saída
`a := 5; b := a^2; a := 'a'; b;`	`a := 5b := 25a := a25`
`x := 3; x := 'x'; x;`	`x := 3x := xx`
`x := 5: 'x'; evaln(x); unassign('x'): x;`	`xxx`
`a, b := 1, 2; a+b;`	`a, b := 1, 23`
`restart; x;`	`x`
`x + x; 2*(x+1);`	`2*x2*x+2`

`restart` limpa variáveis, suposições, pacotes e `Digits`. Aspas simples (`'x'`) suspendem a avaliação; `x := 'x'` desatribui. Nomes com espaço usam crase: `` `a b` ``.

4.2 Procedimentos e funções-seta

```
f := x -> x^2 + 1; f(3); g := (x,y) -> x*y; g(2,5);
f := x -> x^2+1
10
g := (x, y) -> x*y
10
```

```
p := proc(n) local k, s; s := 0; for k from 1 to n do s := s + k end do;
 ↪  return s end proc; p(10);
p := proc(n) local k, s; s := 0; for k from 1 to n do s := s+k end do;
 ↪  return s end proc
55
```

`proc` aceita `local`, `global`, `option remember`, parâmetros tipados (`n::nonnegint`), `args`/`nargs`, `return`, `error` e fecha com `end proc` (ou `end`):

```
fib := proc(n::nonnegint) option remember; if n < 2 then n else fib(n-1) +
 ↪  fib(n-2) end if end proc: fib(50);
12586269025

f := proc() nargs, [args] end proc: f(a, b, c);
```

```
3, [a, b, c]
```

`unapply` transforma uma expressão em função:

```
h := unapply(x^2+y, x); h(3);
h := x -> x^2+y
y+9
```

4.3 Derivada de operador `D`, composição `@` e `@@`

```
f := x -> x^3; D(f); D(f)(2); (D@@2)(f);
f := x -> x^3
x -> 3*x^2
12
x -> 6*x

f := (x,y) -> x^2*y; D[1](f); D[2](f);
f := (x, y) -> x^2*y
(x, y) -> 2*x*y
(x, y) -> x^2

(sin@cos)(x);
sin(cos(x))
```

4.4 Controle de fluxo

```
for i from 10 to 1 by -3 do printf("%d ", i) end do; printf("\n");
10 7 4 1

for x in [1,2,3] do printf("%d ", x^2) end do; printf("\n");
1 4 9

if 3 > 2 then "yes" else "no" end if;
"yes"
```

`while`, `break`, `next`, `elif`, e as formas curtas `fi` e `od` também funcionam. As instruções de um laço no nível superior terminadas em `;` mostram o resultado:

```
for i to 3 do i; end do;
1
2
3
```

4.5 `seq`, `add`, `mul`, `map`, `select`, `zip`, listas

Entrada	Saída
`seq(i^2, i=1..6);`	`1, 4, 9, 16, 25, 36`
`add(i, i=1..100); mul(i, i=1..6);`	`5050720`
`map(x -> x^2, [1,2,3]);`	`[1, 4, 9]`
`select(isprime, [seq(i,i=1..20)]);`	`[2, 3, 5, 7, 11, 13, 17, 19]`

Entrada	Saída
`zip((a,b)->a+b, [1,2,3],[4,5,6]);`	`[5, 7, 9]`
`L := [3,1,2]; sort(L); nops(L); op(2, L); L[2]; subsop(2=9, L);`	`L := [3, 1, 2][1, 2, 3]311[3, 9, 2]`
`{3,1,2} union {5}; {1,2} intersect {2,3}; member(2, [1,2,3]);`	`{1, 2, 3, 5}{2}true`
`x$3; y(i)$i=1..3;`	`x, x, xy(1), y(2), y(3)`
`f~([1,2,3]); sin~([0, Pi/2]);`	`[f(1), f(2), f(3)][0, 1]`
`sum(i^2, i=1..10); add(i^2, i=1..10);`	`385385`

Com limites simbólicos, `add`/`mul` passam para `sum`/`product` do motor. `eval(e, x=a)`, `subs(x=a, e)` (nessa ordem) e `subsop` também existem:

```
eval(x^2+y, x=2); subs(x=2, x^2+y); eval(x^2+y, [x=1,y=2]);
y+4
y+4
3
```

Tabelas e strings: `t := table(): t[a] := 1: t[b] := 2: t[a]+t[b];` dá `3`; `length("abc")` dá `3`; `cat("a", 1)` dá `"a1"`.

4.6 Matrix e Vector com colchetes angulares

Sintaxe	Resultado
`<1,2,3>`	`Vector[column]([1, 2, 3])`
`<1|2|3>`	`Vector[row]([1, 2, 3])`
`<<1|2>,<3|4>>`	`Matrix(2, 2, [[1, 2], [3, 4]])`
`Matrix(2,3,(i,j)->i+j)`	`Matrix(2, 3, [[2, 3, 4], [3, 4, 5]])`
`Vector[row]([1,2])`	`Vector[row]([1, 2])`

```
A := <<1|2>,<3|4>>; A . A; A^(-1); A^2;
A := Matrix(2, 2, [[1, 2], [3, 4]])
Matrix(2, 2, [[7, 10], [15, 22]])
Matrix(2, 2, [[-2, 1], [3/2, -1/2]])
Matrix(2, 2, [[7, 10], [15, 22]])

M := <<1|2>,<3|4>>: map(x->x^2, M); 2*M + M; M . <1,2>;
Matrix(2, 2, [[1, 4], [9, 16]])
Matrix(2, 2, [[3, 6], [9, 12]])
Vector[column]([5, 11])
```

O produto matricial é o ponto `.`; `M[1,2]` indexa.

4.7 Pacotes: `with`, `:-` e `[]`

Há três formas de chamar uma função de pacote, e o nome curto só funciona **depois de `with`**:

```
LinearAlgebra:-Determinant(<<1|2>,<3|4>>);
-2

LinearAlgebra[Transpose](<<1|2>,<3|4>>);
Matrix(2, 2, [[1, 3], [2, 4]])

with(LinearAlgebra): Determinant(<<1|2>,<3|4>>);
-2

Determinant(<<1|2>,<3|4>>);
Determinant(Matrix(2, 2, [[1, 2], [3, 4]]))
```

(A última não resolve o nome curto e devolve a chamada sem avaliar.) Pacotes reconhecidos: `LinearAlgebra`, `inttrans`, `VectorCalculus`, `student`, `numtheory`, `Groebner`, `combinat`, `ListTools`, `DEtools`. Quando dois pacotes exportam o mesmo nome (por exemplo `Basis`, em `LinearAlgebra` e `Groebner`), vale o último carregado com `with`.

4.8 `read`

`read "arquivo.mpl"` executa um arquivo (caminho relativo ao diretório atual):

```
$ cat lib.mpl
sq := x -> x^2:
cube := x -> x^3:
printf("loaded\n"):
$ algebraking -e 'read "lib.mpl"; sq(3) + cube(2);'
loaded
17
```

4.9 `try` / `catch`

```
try 1/0; catch: print("caught"); end try;
"caught"

try error "boom" catch "boom": print("caught boom") end try;
"caught boom"

try error "boom %1", 5 catch: lprint(lasterror) end try;
"boom 5"

r := proc(x) try if x < 0 then error "negative: %1", x end if; sqrt(x)
 ↪  catch: -1 finally print("done") end try end proc; r(4); r(-4);
r := proc(x) try if x < 0 then error "negative: %1", x end if; sqrt(x)
 ↪  catch: -1 finally print("done") end try end proc
"done"
2
"done"
-1
```

Erros fora de `try` aparecem como `Error, mensagem`; no REPL a sessão continua, em script o

programa para.

4.10 `assume`, `additionally`, `assuming`, `about`

```
assume(x>0): x; x^2; about(x);
x
x^2
x:
  is assumed to be: 0 < x

sqrt(x^2) assuming x>0;
x

sqrt(x^2) assuming x::real;
abs(x)

is(x>0) assuming x>1;
true

assume(n, integer): sin(n*Pi); additionally(n>0): about(n);
0
n:
  is assumed to be: n::integer, 0 < n

simplify(sqrt(x^2), assume=positive);
x
```

4.11 `Digits` e `evalf[n]`

```
evalf(Pi); evalf[30](Pi); Digits := 20; evalf(sqrt(2));
3.141592654
3.14159265358979323846264338328
Digits := 20
1.4142135623730950488

evalf(Pi, 50);
3.14159265358979323846264338327950288419716939937751
```

`Digits` vale 10 por padrão; `evalf(e, n)` e `evalf[n](e)` avaliam `e` com `Digits = n`.

4.12 `printf`, `sprintf`, `print`, `lprint`, `latex`

```
printf("%d! = %d\n", 20, 20!); printf("%a is %f\n", x, 3.14159);
↪  printf("%.3e\n", 12345.678);
20! = 2432902008176640000
x is 3.141590
1.235e+04

sprintf("%5.2f|%-4d|%s", 3.14159, 42, "ok");
" 3.14|42  |ok"

latex(x^2/3);
\frac{x^{2}}{3}
```

Especificadores: `%d %a %f %e %g %s`. `kernelopts(version)` devolve `"AlgebraKing 0.0.1"`.

4.13 Precedência e operadores

Precedência dos operadores: `||`, `$`, `!`, `^`/`**` (**não associativo**), − unário, `* /`, `+ −`, `mod`, `..`, relações, `not`, `and`, `or`, `xor`, `implies`, `->`, `,`, `:=`.

```
a^b^c;
Error, ambiguous use of `^`, please use parentheses

1/3 + 1/6; 2^100; 30!; 7 mod 3; 10!/(8!);
1/2
1267650600228229401496703205376
265252859812191058636308480000000
1
90
```

---

5 Matemática

Esta seção lista o que realmente existe no motor, por área, com um exemplo executado. Qualquer nome que não seja da linguagem nem do motor volta sem avaliar (`undefined_fn(3)` → `undefined_fn(3)`). Os resultados são dados em 1-D.

5.1 Álgebra e simplificação

Funções: `simplify` (também `simplify(e, trig)`, `assume=`), `expand`, `factor` (com extensões), `normal`, `combine`, `collect`, `sort`, `rationalize`, `radnormal`, `evalc`, `evalf`, `evalb`, `is`, `testeq`, `convert`.

Entrada	Saída
`expand((x+1)^3);`	`x^3+3*x^2+3*x+1`
`factor(x^3-6*x^2+11*x-6);`	`(x-3)*(x-2)*(x-1)`
`factor(x^2+1, I);`	`(x-I)*(x+I)`
`simplify(sin(x)^2+cos(x)^2);`	`1`
`normal((x^2-1)/(x-1));`	`x+1`
`combine(sin(x)*cos(x), trig);`	`sin(2*x)/2`
`collect(x*y+x*z+y, x);`	`(y+z)*x+y`
`sort(x+x^3+x^2+1, x);`	`x^3+x^2+x+1`
`rationalize(1/(1+sqrt(2)));`	`sqrt(2)-1`
`radnormal(sqrt(8));`	`2*sqrt(2)`
`evalc(exp(I*Pi/3));`	`sqrt(3)*I/2+1/2`
`convert((x^2+1)/(x*(x-1)), parfrac, x);`	`2/(x-1)-1/x+1`
`convert(sin(x), exp);`	`-I*(exp(I*x)-exp(-I*x))/2`
`testeq(sin(x)^2+cos(x)^2, 1);`	`true`
`is(sqrt(2) > 1);`	`true`

Entrada	Saída
`evalb(2+2=4);`	`true`

`convert` aceita `parfrac`, `exp`, `trig`, `sincos`, `tan`, `ln`, `polynom`, `rational`, `list`, `set`, `float`, `radical`, `piecewise`, `Heaviside`, `binomial`, `factorial`, `GAMMA` (exemplos: `convert(0.75, rational)` → `3/4`; `convert([1,2,3], set)` → `{1, 2, 3}`).

5.2 Polinômios

Funções: `coeff`, `coeffs`, `degree`, `ldegree`, `lcoeff`, `tcoeff`, `numer`, `denom`, `quo`, `rem`, `gcd`, `lcm`, `resultant`, `discrim`, `content`, `primpart`, `sqrfree`, `roots`, `RootOf`/`allvalues`, `interp`, `Groebner[Basis]`.

Entrada	Saída
`coeff(3*x^2+2*x+1, x, 2); degree(x^5+x, x);`	`35`
`coeffs(x^2+3*x+1, x);`	`1, 3, 1`
`quo(x^3+1, x+1, x); rem(x^3+2, x+1, x);`	`x^2-x+11`
`gcd(x^2-1, x^2+2*x+1); lcm(x^2-1, x+3);`	`x+1x^3+3*x^2-x-3`
`resultant(x^2-1, x-2, x);`	`3`
`discrim(a*x^2+b*x+c, x);`	`-4*a*c+b^2`
`sqrfree(x^3-x^2-x+1);`	`[1, [[x+1, 1], [x-1, 2]]]`
`content(6*x^2+9*x, x); primpart(6*x^2+9*x, x);`	`32*x^2+3*x`
`roots(x^2-5*x+6);`	`[[3, 1], [2, 1]]`
`interp([1,2,3],[1,4,9], x);`	`x^2`
`Groebner[Basis]([x^2+y, x*y-1], plex(x,y));`	`[y^2+x, y^3+1]`
`allvalues(RootOf(x^2-2, x));`	`sqrt(2), -sqrt(2)`

5.3 Teoria dos números e funções inteiras

Funções: `ifactor`, `ifactors`, `isprime`, `ithprime`, `nextprime`, `prevprime`, `igcd`, `ilcm`, `iquo`, `irem`, `isqrt`, `iroot`, `mod`, `modp`, `mods`, `numtheory[phi]`, `sigma`, `tau`, `mobius`, `divisors`, `binomial`, `factorial`, `floor`, `ceil`, `round`, `trunc`, `frac`, `abs`, `signum`, `max`, `min`, `fibonacci`, `lucas`, `bernoulli`, `euler`, `harmonic`, `pochhammer`, `doublefactorial`.

Entrada	Saída
`ifactor(360); ifactors(360);`	`2^3*3^2*5[1, [[2, 3], [3, 2], [5, 1]]]`
`isprime(97); ithprime(10);` `nextprime(100); prevprime(100);`	`true2910197`
`igcd(12, 18); ilcm(4, 6);` `iquo(17, 5); irem(17, 5);`	`61232`

`numtheory[phi](36);`
`numtheory[sigma](12);`
`numtheory[tau](12);`
`numtheory[mobius](30);`
	`12286-1`

Entrada	Saída
`numtheory[divisors](12);`	`{1, 2, 3, 4, 6, 12}`
`binomial(10,3); factorial(6);`	`120720`
`floor(2.7); ceil(2.1); round(2.5); trunc(-2.7); frac(2.75);`	`233-20.75`
`max(1,5,3); min(4,2); isqrt(50); iroot(27, 3);`	`5273`
`with(combinat): fibonacci(10);`	`55`
`lucas(5); bernoulli(4); euler(4); harmonic(5);`	`11-1/305137/60`
`pochhammer(3, 2); doublefactorial(5);`	`1215`

Nota: `fibonacci` pertence a `combinat` (`combinat[fibonacci](10)` ou `with(combinat);` sem isso a chamada volta sem avaliar). Depois de `with(numtheory)`, `phi(36)` e `divisors(12)` funcionam sem prefixo.

5.4 Cálculo

Funções: `diff`, `D`, `int`, `Int`, `value`, `limit`, `series`, `taylor`, `mtaylor`, `sum`, `product`, `Sum`, `Product`, `Limit`, `Diff`, `residue`, `singular`, `implicitdiff`, `extrema`, `maximize`, `minimize`.

Entrada	Saída
`diff(sin(x)*x, x);`	`x*cos(x)+sin(x)`
`diff(x^4, x$2);`	`12*x^2`
`diff(x^2*y^3, x, y);`	`6*x*y^2`
`int(x^2, x);`	`x^3/3`
`int(x^2, x=0..3);`	`9`
`int(exp(-x^2), x=-infinity..infinity);`	`sqrt(Pi)`
`int(sin(x)^2, x=0..Pi);`	`Pi/2`
`Int(x^2,x); value(Int(x^2,x));`	`Int(x^2, x)x^3/3`
`limit(sin(x)/x, x=0);`	`1`
`limit(1/x, x=0, right);`	`infinity`
`limit(abs(x)/x, x=0, left);`	`-1`
`limit((1+1/n)^n, n=infinity);`	`exp(1)`
`series(exp(x), x=0, 5);`	`1+x+x^2/2+x^3/6+x^4/24+O(x^5)`
`taylor(cos(x), x=0, 6);`	`1-x^2/2+x^4/24+O(x^6)`
`mtaylor(sin(x)*cos(y), [x,y], 4);`	`-x^3/6-x*y^2/2+x`
`sum(1/k^2, k=1..infinity);`	`Pi^2/6`
`sum(k, k=1..n);`	`n^2/2+n/2`
`product(k, k=1..5);`	`120`
`residue(1/(x^2+1), x=I);`	`-I/2`
`singular(1/(x*(x-1)));`	`{x = 0, x = 1}`
`implicitdiff(x^2+y^2=1, y, x);`	`-x/y`

Entrada	Saída
`extrema(x^3-3*x, {}, x);`	`{-2, 2}`
`maximize(-x^2+2*x, x); minimize(x^2-2*x, x);`	`1-1`

5.5 Resolução de equações

Funções: `solve`, `fsolve`, `isolve`, `rsolve`, `dsolve`, `pdsolve`, `odetest`.

Entrada	Saída
`solve(x^2-5*x+6=0, x);`	`3, 2`
`solve({x+y=3, x-y=1}, {x,y});`	`{x = 2, y = 1}`
`solve(x^3=1, x);`	`1, sqrt(3)*I/2-1/2, -sqrt(3)*I/2-1/2`
`solve(x^2>4, x);`	`RealRange(-infinity, Open(-2)), RealRange(Open(2), infinity)`
`solve(a*x^2+b*x+c=0, x);`	`(-b-sqrt(-4*a*c+b^2))/(2*a), (-b+sqrt(-4*a*c+b^2))/(2*a)`
`fsolve(cos(x)=x, x);`	`0.7390851332`
`isolve(2*x+3*y=7);`	`{x = 3*_Z1-7, y = -2*_Z1+7}`
`rsolve({f(n+1)=2*f(n), f(0)=1}, f(n));`	`2^n`
`dsolve(diff(y(x),x)=-2*y(x), y(x));`	`y(x) = _C1*exp(-2*x)`
`dsolve({diff(y(x),x,x)+y(x)=0, y(0)=0, D(y)(0)=1}, y(x));`	`y(x) = sin(x)`
`dsolve({diff(x(t),t)=y(t), diff(y(t),t)=-x(t)}, {x(t),y(t)});`	`{x(t) = _C1*sin(t)+_C2*cos(t), y(t) = _C1*cos(t)-_C2*sin(t)}`
`odetest(y(x)=exp(-2*x)*_C1, diff(y(x),x)=-2*y(x));`	`0`
`pdsolve(diff(u(x,t),x)+diff(u(x,t),t)=0, u(x,t));`	`u(x, t) = _F1(-t+x)`

5.6 Álgebra linear (`LinearAlgebra`)

Funções (use `with(LinearAlgebra)` ou o prefixo `LinearAlgebra:-`): `Matrix`, `Vector`, `Determinant`, `MatrixInverse`, `Transpose`, `HermitianTranspose`, `Eigenvalues`, `Eigenvectors`, `Rank`, `Trace`, `NullSpace`, `ColumnSpace`, `RowSpace`, `LinearSolve`, `ReducedRowEchelonForm`, `GaussianElimina` `LUDecomposition`, `QRDecomposition`, `CharacteristicPolynomial`, `MinimalPolynomial`, `JordanForm`, `DotProduct`, `CrossProduct`, `Norm`, `Normalize`, `IdentityMatrix`, `ZeroMatrix`, `DiagonalMatrix`, `MatrixExponential`, `MatrixPower`, `IsDefinite`, `GramSchmidt`, `SingularValues`.
Aritmética: `A.B`, `A^(-1)`, `A+B`, `k*A`.

Com `A := <<2|1>,<1|3>>` e `with(LinearAlgebra):`

Entrada	Saída
`Determinant(A);`	`5`

Entrada	Saída
`MatrixInverse(A);`	`Matrix(2, 2, [[3/5, -1/5], [-1/5, 2/5]])`
`Trace(A); Rank(A);`	`52`
`Eigenvalues(A);`	`Vector[column]([-sqrt(5)/2+5/2, sqrt(5)/2+5/2])`
`Eigenvectors(A);`	`Vector[column]([-sqrt(5)/2+5/2, sqrt(5)/2+5/2]), Matrix(2, 2, [[-sqrt(5)/2-1/2, sqrt(5)/2-1/2], [1, 1]])`
`Eigenvalues(evalf(A));`	`Vector[column]([1.38196601125011, 3.61803398874989])`

Com `B := <<1|2>,<3|4>>`:

Entrada	Saída
`CharacteristicPolynomial(B, x);`	`x^2-5*x-2`
`LinearSolve(B, <5,6>);`	`Vector[column]([-4, 9/2])`
`ReducedRowEchelonForm(B);`	`Matrix(2, 2, [[1, 0], [0, 1]])`
`QRDecomposition(B);`	`Matrix(2, 2, [[sqrt(10)/10, 3*sqrt(10)/10], [3*sqrt(10)/10, -sqrt(10)/10]]), Matrix(2, 2, [[sqrt(10), 7*sqrt(10)/5], [0, sqrt(10)/5]])`
`NullSpace(<<1|2>,<2|4>>);`	`{Vector[column]([-2, 1])}`
`MatrixExponential(<<0|1>,<0|0>>, t);`	`Matrix(2, 2, [[1, t], [0, 1]])`
`JordanForm(<<4|1>,<2|3>>);`	`Matrix(2, 2, [[2, 0], [0, 5]])`
`DotProduct(<1,2,3>,<4,5,6>);`	`32`
`CrossProduct(<1,0,0>,<0,1,0>);`	`Vector[column]([0, 0, 1])`
`Normalize(<3,4>, 2);`	`Vector[column]([3/5, 4/5])`
`GramSchmidt([<1,1>,<1,0>]);`	`[Vector[column]([1, 1]), Vector[column]([1/2, -1/2])]`
`SingularValues(<<3|0>,<0|4>>);`	`Vector[column]([4, 3])`
`IsDefinite(<<2|0>,<0|3>>, query='positive_definite');`	`true`

Matrizes de ponto flutuante usam um resolvedor numérico (mpmath) com no mínimo 15 dígitos.

5.7 Transformadas (`inttrans`)

Funções: `laplace`, `invlaplace`, `fourier`, `invfourier`, `mellin` (com `with(inttrans)`):

Entrada	Saída
`laplace(sin(t), t, s);`	`1/(s^2+1)`
`invlaplace(1/(s^2+1), s, t);`	`sin(t)`
`fourier(exp(-t^2), t, w);`	`sqrt(Pi)*exp(-w^2/4)`
`invfourier(exp(-w^2/4)*sqrt(Pi), w, t);`	`exp(-t^2)`
`mellin(exp(-x), x, s);`	`GAMMA(s)`

5.8 Cálculo vetorial (`VectorCalculus`)

Funções: `Gradient`, `Divergence`, `Curl`, `Laplacian`, `Jacobian`, `Hessian`, `Wronskian` (com `with(VectorCalculus)`):

Entrada	Saída
`Gradient(x^2+y^2, [x,y]);`	`Vector[column]([2*x, 2*y])`
`Divergence(<x^2,y^2>, [x,y]);`	`2*x+2*y`
`Curl(<-y,x,0>, [x,y,z]);`	`Vector[column]([0, 0, 2])`
`Laplacian(x^2+y^2, [x,y]);`	`4`
`Jacobian([x*y, x+y], [x,y]);`	`Matrix(2, 2, [[y, x], [1, 1]])`
`Hessian(x^2*y, [x,y]);`	`Matrix(2, 2, [[2*y, 2*x], [2*x, 0]])`
`Wronskian([sin(x),cos(x)], x);`	`Matrix(2, 2, [[sin(x), cos(x)], [cos(x), -sin(x)]])`

5.9 Funções elementares e especiais

Elementares: `sin cos tan cot sec csc`, `sinh ... csch`, `arcsin ... arccsc`, `arcsinh ... arccsch`, `exp`, `ln`, `log[b]`, `log10`, `log2`, `sqrt`, `root`, `surd`, `abs`, `signum`, `csgn`, `argument`, `Re`, `Im`, `conjugate`.

Especiais: `GAMMA`, `Beta`, `Psi`, `Zeta`, `erf`, `erfc`, `erfi`, `erfinv`, `Ei`, `Si`, `Ci`, `Shi`, `Chi`, `Li`, `FresnelS`, `FresnelC`, `LambertW`, `BesselJ/Y/I/K`, `AiryAi/Bi`, `LegendreP`, `HermiteH`, `LaguerreL`, `ChebyshevT/U`, `GegenbauerC`, `JacobiP`, `hypergeom`, `EllipticK/E/F/Pi`, `dilog`, `polylog`, `Heaviside`, `Dirac`, `piecewise`.

Entrada	Saída
`GAMMA(1/2); Beta(2,3); Psi(1); Zeta(2);`	`sqrt(Pi)1/12-gammaPi^2/6`
`erf(1.0); Ei(1.0); Si(1.0); LambertW(1.0);`	`0.84270079291.8951178160.94608307040.5671432904`
`BesselJ(0, 1.0); AiryAi(0.5);`	`0.76519768660.2316936065`
`LegendreP(2, x); HermiteH(3, x); LaguerreL(2, x);`	`3*x^2/2-1/28*x^3-12*xx^2/2-2*x+1`
`ChebyshevT(3, x); ChebyshevU(2, x);`	`4*x^3-3*x4*x^2-1`
`EllipticK(0.5); EllipticE(0.5);`	`1.6857503551.467462209`

Entrada	Saída
`dilog(2.0); polylog(2, 0.5);`	`-0.82246703340.5822405265`
`arcsin(1/2); arctan(1, 1); log[2](8); log10(1000);`	`Pi/6Pi/433`
`sqrt(18); surd(-8, 3); root(16, 4);`	`3*sqrt(2)-22`
`csgn(-1+I); argument(1+I); Re(3+4*I); conjugate(3+4*I);`	`-1Pi/433-4*I`
`Heaviside(-1);`	`0`
`piecewise(x<0, -x, x);`	`piecewise(x < 0, -x, x)`

`hypergeom([1,2],[3],x)` volta sem avaliar (só avalia em certos casos).

5.10 Diversos: tipos e estrutura

`type`, `whattype`, `has`, `indets`, `lhs`, `rhs`, `op`, `nops`, `subs`, `eval`, `unapply`.

Entrada	Saída
`type(3, integer); type(x, polynom); type([1,2], list);`	`truetruetrue`
`whattype(2.5);`	`float`
`indets(x^2+y*z); has(x^2+y, y);`	`{x, y, z}true`
`lhs(a=b); rhs(a=b);`	`ab`
`op(x+y); nops(x+y+z);`	`x, y3`
`op(0, f(x,y));`	`f`

---

6 Usando com o Claude (MCP)

MCP (Model Context Protocol) é o protocolo pelo qual um assistente como o Claude chama ferramentas externas. O `algebraking mcp` é um servidor MCP que conversa por stdin/stdout (JSON-RPC 2.0): o Claude envia o passo da sua conta e recebe o veredito.

6.1 Registro automático

O `./install.sh` já faz o registro (se o comando `claude` existir). Para refazer:

```
./install.sh
```

6.2 Registro manual

```
claude mcp add algebraking -s user -- "$(command -v algebraking)" mcp
```

(o escopo `-s user` vale para todos os seus projetos). Verificar:

```
claude mcp list              # algebraking deve aparecer como conectado
claude mcp get algebraking   # mostra comando, argumentos e estado
```

Dentro do Claude Code, o comando `/mcp` mostra os servidores e as ferramentas. Sessões já abertas precisam ser reiniciadas para carregar um servidor recém-registrado.

6.3 Claude Desktop

Edite `~/.config/Claude/claude_desktop_config.json` (Linux) e use o caminho **absoluto** do binário (o JSON não expande `~`):

```
{
  "mcpServers": {
    "algebraking": {
      "command": "/home/SEU_USUARIO/.local/bin/algebraking",
      "args": ["mcp"]
    }
  }
}
```

Reinicie o Claude Desktop depois de salvar.

6.4 As 8 ferramentas

Todas aceitam a sintaxe do AlgebraKing. Os argumentos com `?` são opcionais. `session?` escolhe uma sessão nomeada (padrão `"main"`). Todas as ferramentas de conferência aceitam `field?: real` (padrão, porque contas de física e química usam variáveis reais) ou `complex`.

Ferramenta	Argumentos	O que faz
`algebra_eval`	`code`, `session?`	executa instruções do AlgebraKing numa sessão persistente e devolve cada resultado em 1-D, mais o LaTeX do último
`verify_equal`	`lhs`, `rhs`, `assumptions?`, `field?`, `session?`	decide se `lhs = rhs` no domínio comum: veredito, método, condições, domínio de cada lado, diferença, contraexemplo dentro do domínio, sugestão de hipótese e limitações
`check_derivative`	`f`, `derivative`, `var` (+ `assumptions?`, `field?`, `session?`)	confere se `diff(f, var)` é a derivada declarada
`check_integral`	`integrand`, `antiderivative`, `var` (+ `assumptions?`, `field?`, `session?`)	confere se `diff(antiderivative, var)` é o integrando (primitiva correta a menos de constante)

Ferramenta	Argumentos	O que faz
`check_solution`	`equations`, `solution` (+ `assumptions?`, `field?`, `session?`)	confere cada candidata na equação **original** e no seu domínio, lista as rejeitadas e diz se as aceitas são todas as soluções
`check_step`	`before`, `after`, `var?` (+ `assumptions?`, `field?`, `session?`)	classifica um passo de resolução: equivalente, implica (soluções espúrias), é implicado (soluções perdidas), não equivalente ou indeterminado
`check_ode_solution`	`ode`, `solution`, `func` (+ `assumptions?`, `field?`, `session?`)	como `odetest`: substitui na EDO e nas condições iniciais
`algebra_reset`	`session?`	`restart` da sessão

`assumptions` é uma string como `"x>0, n::integer"`. Em `check_solution`, `equations` pode ser uma equação ou um conjunto (`{x+y=3, x-y=1}`); uma expressão `e` significa `e = 0`; `solution` é uma candidata (`x=2`, `{x=2, y=1}`) ou várias (`[x=0, x=3]`, `[{x=1, y=2}, {x=2, y=1}]`). Em `check_ode_solution`, a EDO pode vir num conjunto com condições iniciais.

6.4.1 Campos da resposta

A primeira linha é sempre `verdict: equal | not_equal | undetermined`, mas o veredito sozinho não basta. Leia também:

Campo	Significado
`method`	`symbolic` (prova), `numeric` (amostragem), `convergence`, `solveset`, `domain`
`field`	corpo usado: `real` ou `complex`
`conditions`	condições de existência do domínio comum, na sintaxe do AlgebraKing (`x <> 1`, `abs(x) < 1`, `sin(x) <> 0`). `equal` vale **só** onde elas valem
`domain`	domínio de cada lado, o comum e se os dois lados têm o mesmo domínio natural (`same domain: no, they differ where x = 1`)
`counterexample`	só aparece com ponto **dentro** do domínio comum, com os valores de cada lado
`outside the common real domain`	ponto em que um lado é real e está no seu domínio, e o outro não é real (por exemplo `2*ln(x)` com `x < 0`)
`suggestion`	hipótese sob a qual a igualdade foi **provada simbolicamente** (`equal if x >= 0`); não muda o veredito

Campo	Significado
`note` / `limitation`	explicações e o que não foi provado (amostragem, convergência, conjuntos que o `solveset` não resolveu)
`assumptions used`	as hipóteses da sessão e do argumento `assumptions`

`sum`, `product` e `int` chegam ao motor **inertes** (`Sum`, `Product`, `Int`) nas ferramentas de conferência, para que a condição de convergência não se perca; em `algebra_eval` e no REPL continuam devolvendo a forma fechada.

6.4.2 Exemplos de pedido e resposta real

`verify_equal` — *"Confere se (x^2-1)/(x-1) é igual a x+1."*

```
verdict: equal
method: symbolic
field: real
conditions: x <> 1 (the result holds only where these hold)
domain: lhs: x <> 1; rhs: all real numbers; common: x <> 1; same domain:
↪  no, they differ where x = 1
difference lhs - rhs: 0
note: equal where both sides are defined; the domains differ where x = 1
lhs: (x^2-1)/(x-1)
rhs: x+1
```

"sqrt(x^2) = x?" — sem hipótese o contraexemplo está dentro do domínio, e a ferramenta sugere a hipótese que torna a identidade verdadeira:

```
verdict: not_equal
method: numeric
field: real
conditions: none
domain: lhs: all real numbers; rhs: all real numbers; common: all real
↪  numbers; same domain: yes
difference lhs - rhs: -x+abs(x)
counterexample: x = -2.1 (inside the common domain); lhs = 2.1, rhs = -2.1
suggestion: equal if x >= 0 -- found by trying a fixed menu of assumptions;
↪  the equality under it is a symbolic proof
note: the counterexample lies inside the common domain
```

Com `assumptions: "x>0"` o resultado é `equal`, método `symbolic`, com `assumptions used: x > 0`.

"ln(x^2) = 2 ln(x)?" — no domínio comum (`x > 0`) os lados coincidem, mas para `x < 0` o lado esquerdo é real e o direito não:

```
verdict: not_equal
method: numeric
field: real
conditions: x > 0 (the result holds only where these hold)
```

```
domain: lhs: x <> 0; rhs: x > 0; common: x > 0; same domain: no, they differ
↪  where x < 0
difference lhs - rhs: -2*ln(x)+ln(x^2)
outside the common real domain: at x = -2.1 lhs = 1.483874689, rhs =
↪  1.483874689+6.283185307*I
suggestion: equal if x > 0 -- found by trying a fixed menu of assumptions;
↪  the equality under it is a symbolic proof
note: the sides agree on the common real domain, but at x = -2.1 the lhs is
↪  real and in its domain while the rhs is not real (outside its real
↪  domain) and has a different value
note: over the reals the rhs requires x > 0, the lhs only x <> 0
```

A identidade válida é `ln(x^2) = 2*ln(abs(x))`: sai `equal`, `conditions: x <> 0`, `same domain: yes`.

"sum(x^n, n=0..infinity) = 1/(1-x)?" — a série só converge para `abs(x) < 1`:

```
verdict: equal
method: symbolic
field: real
conditions: abs(x) < 1 (the result holds only where these hold)
domain: lhs: abs(x) < 1; rhs: x <> 1; common: abs(x) < 1; same domain: no,
↪  they differ where x <= -1 or x > 1
difference lhs - rhs: 0
note: Sum(x^n, n = 0 .. infinity) = 1/(1-x) only where abs(x) < 1
↪  (convergence condition)
note: Sum(x^n, n = 0 .. infinity) diverges at x = -1 (boundary), x = 1
↪  (boundary), x = -2, x = 2
note: equal where both sides are defined; the domains differ where x <= -1 or
↪  x > 1
limitation: divergence outside abs(x) < 1 was checked only at the listed
↪  points, not proved
lhs: Sum(x^n, n = 0 .. infinity)
rhs: 1/(-x+1)
```

Identidades trigonométricas com polos trazem a condição: `(1-cos(x)^2)/sin(x)^2 = 1` sai `equal` com `conditions: sin(x) <> 0` e a nota `sin(x) <> 0: excludes x = Pi*_Z1 (_Z1 integer)` (`_Z1` é um inteiro qualquer).

`check_step` — *"Confere o passo: de sqrt(x+1) = x-1 elevei ao quadrado e cheguei em x+1 = (x-1)^2."* (`before: "sqrt(x+1) = x-1"`, `after: "x+1 = (x-1)^2"`)

```
classification: implies (every solution of before solves after, but after
↪  has extra (spurious) solutions)
solutions before: x = 3
solutions after: x = 0, x = 3
spurious solutions (added by the step): x = 0
verdict: not_equal
method: solveset
field: real
conditions: x >= -1 (domain of the equations)
domain: before: x >= -1; after: all real numbers
note: the new equation is the old one squared (lhs^2 = rhs^2); squaring
↪  adds the solutions of lhs = -rhs, so those must be rejected by checking
↪  them in the original equation
before: sqrt(x+1) = x-1
after: x+1 = (x-1)^2
```

Dividir pela incógnita perde soluções (`x^2 = x` para `x = 1`):

```
classification: implied_by (every solution of after solves before, but
↪  solutions of before are lost)
solutions before: x = 0, x = 1
solutions after: x = 1
lost solutions: x = 0
...
note: the step divides by x, which vanishes at solutions of the original
↪  equation; those solutions are lost
```

Famílias infinitas aparecem como famílias, nunca como listas finitas: de `sin(x) = 0` para `x = 0` sai `implied_by` com `solutions before: x = Pi*_Z1 (_Z1 integer)` e `lost solutions: x = Pi*_Z1 (_Z1 integer) except x = 0`. Quando o `solveset` não resolve uma das equações (devolve `ConditionSet`, como em `exp(x) = x+2`), a classificação é `undetermined`, com o motivo. Com duas expressões (sem =), o passo é uma reescrita e é conferido como no `verify_equal`, com domínios. Parâmetros além de `var` são tratados como genéricos pelo `solveset`; o relatório avisa, e um fator só dos parâmetros aparece na nota (`a*x = a*b` para `x = b` perde todo x quando `a = 0`).

`check_derivative` — *"Confere se a derivada de x*exp(x) é exp(x)."*

```
The claimed derivative is WRONG.
computed diff(f, x): x*exp(x)+exp(x)
verdict: not_equal
method: numeric
field: real
conditions: none
domain: lhs: all real numbers; rhs: all real numbers; common: all real
↪  numbers; same domain: yes
difference lhs - rhs: x*exp(x)
counterexample: x = -2.1 (inside the common domain); lhs = -0.1347020711,
↪  rhs = 0.1224564283
note: the counterexample lies inside the common domain
```

E com `(x+1)*exp(x)`: `The claimed derivative is correct.` (`verdict: equal`, `method: symbolic`).

`check_integral` — *"Confere se uma primitiva de x*cos(x) é x*sin(x)+cos(x)."*

```
The antiderivative is correct (up to an additive constant).
diff(antiderivative, x): x*cos(x)
verdict: equal
method: symbolic
field: real
conditions: none
domain: lhs: all real numbers; rhs: all real numbers; common: all real
↪  numbers; same domain: yes
difference lhs - rhs: 0
```

`check_solution` — *"Confere se x=0 e x=3 resolvem sqrt(x+1) = x-1."* (`equations: "sqrt(x+1) = x-1"`, `solution: "[x=0, x=3]"`)

```
Some candidates are REJECTED: x = 0.
equation 1: sqrt(x+1) = x-1
candidate x = 0: rejected
```

```
  equation 1: not satisfied: lhs = 1, rhs = -1
candidate x = 3: accepted
  equation 1: satisfied: lhs - rhs simplifies to 0
all solutions over the reals: x = 3
completeness: the accepted candidates are all the solutions over the reals
verdict: not_equal
method: symbolic
field: real
conditions: x >= -1 (domain of the equations)
```

Uma candidata fora do domínio é rejeitada mesmo que a conta "feche" depois de simplificar: `x = 1` em `(x^2-1)/(x-1) = 2` dá `outside domain: the condition x-1 <> 0 of the original equation fails.` Para `sin(x) = 0` com `[x=0, x=Pi]`, as duas são aceitas, mas a completude diz `the equation has infinitely many solutions over the reals (x = Pi*_Z1 (_Z1 integer)); a finite list of candidates cannot be complete.` Sistema `{x+y=3, x-y=1}` com `{x=2, y=1}`: `The solution satisfies every equation.` e `completeness: the accepted candidates are all the solutions of the polynomial system.`

`check_ode_solution` — *"Confere se y(x)=3 sin(x)+2 cos(x) resolve y' '+y=0."* (`ode: "diff(y(x),x,x)+y(x`
`solution: "y(x)=3*sin(x)+2*cos(x)"`, `func: "y(x)"`)

```
The solution satisfies the ODE (residual 0).
condition 1: diff(y(x), x, x)+y(x) = 0
  after substitution: 0 = 0
  verdict: equal
  method: symbolic
  field: real
  conditions: none
  domain: lhs: all real numbers; rhs: all real numbers; common: all real
  ↪  numbers; same domain: yes
  difference lhs - rhs: 0
```

`algebra_eval` — *"Calcula a derivada de x^2 sin(x) no AlgebraKing."* (`code: "f := x^2*sin(x): diff(f, x);"`)

```
x^2*cos(x)+2*x*sin(x)
LaTeX: x^{2} \cos{\left(x \right)} + 2 x \sin{\left(x \right)}
```

`algebra_reset` — *"Limpa a sessão."* → `Session reset.`

6.5 Como o veredito funciona

`verify_equal` (e as ferramentas `check_derivative`, `check_integral`, `check_ode_solution` e `check_step` com duas expressões, que a usam por baixo) segue este roteiro, sobre o corpo pedido (`real` por padrão):

1. Para listas, conjuntos, equações e matrizes, compara elemento a elemento (sem análise de domínio; o relatório avisa).
2. `Sum`, `Product` e `Int` são calculados guardando a condição do SymPy (`abs(x) < 1`); a convergência na fronteira e em dois pontos de fora é testada e relatada. Série divergente não tem valor; série cuja convergência não se decide nunca dá `equal`.
3. Coleta as condições de existência de cada lado: denominadores `<> 0`, argumento de `ln` (`> 0` nos reais, `<> 0` nos complexos), raízes e potências fracionárias (`>= 0` nos reais), polos de `tan`, `cot`, `sec`, `csc`, faixas de `arcsin`, `arccos`, `arctanh` etc. Condições já garantidas pelas hipóteses

somem. Os conjuntos excluídos vêm do `solveset`; quando ele não resolve, a condição fica simbólica e uma limitação avisa.
4. Tenta provar `lhs - rhs = 0` simbolicamente no domínio comum (as condições simples, como `x > 0`, viram hipóteses). Zero → **`equal`**, método **`symbolic`**.
5. Teste numérico: pontos aleatórios a 40 dígitos, **só** onde os dois lados existem (e são reais, no corpo real) e satisfazem as condições. Um ponto claramente diferente → **`not_equal`** com esse ponto como contraexemplo. Um ponto em que um lado é real e o outro não → **`not_equal`** explicado (`outside the common real domain`). Todos iguais → `equal` numérico.
6. `equal` significa igual no domínio comum; se os domínios diferem (`x = 1` excluído só à esquerda, por exemplo), o relatório diz onde.
7. Se o veredito não é `equal` e há no máximo 2 variáveis, tenta um cardápio fixo de hipóteses (`x >= 0`, `x > 0`, `x < 0`, `x::integer`) e, se alguma der prova simbólica, mostra `suggestion`. O veredito principal não muda.
8. Caso contrário, **`undetermined`**. O programa nunca levanta erro por motivo matemático.

O que é prova e o que é heurística. É prova: `equal` com método `symbolic`, sob as `conditions` listadas e as hipóteses declaradas; a `suggestion` também é prova simbólica sob a hipótese sugerida, mas a escolha da hipótese é heurística (um cardápio fixo, não uma busca completa). É heurística: `equal` numérico (pontos de amostra), a divergência fora da condição de uma série (testada em pontos), a comparação de domínios com duas ou mais variáveis e a classificação de `check_step` quando há parâmetros (o `solveset` os trata como genéricos). O `check_step` e a completude do `check_solution` dependem do `solveset`: quando ele devolve `ConditionSet`, a resposta é `undetermined` / *not established*, nunca um palpite. Por fim, a linguagem simplifica automaticamente `x/x` para `1` e `sqrt(x)^2` para `x`; a ferramenta recupera essas condições lendo a entrada antes da avaliação, mas não quando a expressão vem de uma variável já atribuída na sessão.

`equal` numérico é evidência forte, não prova. Exemplo real: `erf(x)+erfc(x) = 1` sai `equal` por método `numeric`, e está certo. Mas `sin(x)*Heaviside(x)` contra `abs(sin(x))*Heaviside(x)` também sai `equal` numérico, porque os pontos de teste ficam em [-3, 3] e a diferença só aparece para $x > \pi$ (em `x = 4` as duas expressões valem `sin(4)` e `−sin(4)`). Desconfie de identidades com funções definidas por partes, ramos ou comportamento em regiões distantes. `symbolic` + `equal` é a única resposta que vale como demonstração.

A função `verify(a, b)` da linguagem (REPL e scripts) continua devolvendo só `true`, `false` ou `FAIL`, com o algoritmo antigo (sem corpo nem domínio).

6.6 Sessões

Cada processo do servidor mantém sessões nomeadas, cada uma com seu interpretador: variáveis, `Digits`, suposições e pacotes carregados persistem entre chamadas na mesma sessão. Sem `session`, usa-se `"main"`. Duas sessões não se enxergam:

```
algebra_eval  code="g := 3;"  session="s2"   ->  g := 3
algebra_eval  code="g + 1;"   session="s2"   ->  4
```

`algebra_reset` limpa uma sessão. Fechar o Claude Code encerra o servidor e descarta as sessões.

6.7 Variáveis de ambiente

Variável	Efeito
`ALGEBRAKING_TIMEOUT`	segundos por chamada ao motor (padrão 60). Estourou: o motor é morto, reiniciado e o erro é `Error, (in nome) computation timed out after N s`; as variáveis e suposições da sessão continuam
`ALGEBRAKING_PYTHON`	executável Python do motor (padrão `python3`); útil se o SymPy está em outro Python/venv
`ALGEBRAKING_DEBUG`	qualquer valor: repassa o `stderr` do motor (linhas `[engine] ...`) e imprime tracebacks Python de falhas internas

No registro MCP, passe variáveis com `claude mcp add algebraking -s user -e ALGEBRAKING_TIMEO -- ...` ou na chave `env` do JSON do Claude Desktop.

---

7 Limitações conhecidas

Lista honesta do que o AlgebraKing faz de modo particular ou não faz. O objetivo é resposta correta primeiro; o formato da saída é secundário.

Tema	Comportamento do AlgebraKing
`solve`	devolve todas as soluções que encontra, também as de vários ramos: `solve(sin(x)=0, x)` → `Pi, 0`
Ordem das soluções	`solve(x^2-5*x+6=0, x)` → `3, 2`
Ordem de termos e fatores	vem da forma interna do SymPy: `(x-3)*(x-2)*(x-1)`, `-sqrt(5)/2+5/2`
Formato de ponto flutuante	`0.1362`; `1.5*10^20`
Nomes com suposição	`assume(x>0)` não marca o nome: `x` aparece como `x`
`Heaviside(0)`	devolve `1/2`
Últimos dígitos	resultados de ponto flutuante de 15 dígitos (matrizes numéricas, `HFloat`) podem variar nos dois últimos dígitos

Funções e pacotes que **não estão disponíveis** (a chamada volta sem avaliar ou dá erro de sintaxe): `inttrans`:-`hilbert`, `LegendreQ`, o pacote `Student` (`Student[Calculus1]`), `isolate` (volta sem avaliar), `DEtools` gráficos. Também **não há**: gráficos (`plot`, `plots`), interface gráfica, worksheets `.mw`, geração de código e as bibliotecas `Statistics`/`StringTools` (o nome do pacote é reconhecido, mas sem funções).

A saída em 1-D tem formato próprio, estável entre execuções.

---

8 Solução de problemas

`sympy/mpmath` não encontrado. Sintoma típico:

```
Error, (in int) the math engine stopped:   File ".../engine-....py", line
↪  29, in <module> |     import mpmath | ModuleNotFoundError: No module
↪  named 'mpmath'
```

Instale: `python3 -m pip install --user sympy mpmath` (ou rode `./install.sh`). Se o SymPy está num Python diferente do `python3` do `PATH`, aponte-o com `ALGEBRAKING_PYTHON=/caminho/python`. Não passe `-I` nem `-s` ao Python: o SymPy mora no `site-packages` do usuário.

Motor não inicia. `ALGEBRAKING_PYTHON` errado:

```
Error, cannot start the math engine (/nao/existe): No such file or directory
↪  (os error 2)
```

`cargo: command not found`. Rode `./install.sh` (ele instala o rustup) ou carregue o ambiente com `source ~/.cargo/env` e confira `cargo --version`.

`algebraking: command not found`. `~/.local/bin` não está no `PATH`. Acrescente ao `~/.bashrc` e abra um novo terminal:

```
export PATH="$HOME/.local/bin:$PATH"
```

Servidor MCP aparece como `failed` / não conecta.

1. `claude mcp get algebraking`: confira o comando registrado (caminho absoluto do binário
   - `mcp`) e se o arquivo existe (`ls -l` no caminho; o link quebra se o repositório mudou de lugar).
2. Teste o servidor à mão (deve responder uma linha JSON com `serverInfo`):

   ```
   printf '%s\n' '{"jsonrpc":"2.0","id":1,"method":"initialize","params
   ↪  ":{"protocolVersion":"2025-06-18","capabilities":{},"clientInfo"
   ↪  :{"name":"t","version":"0"}}}' | algebraking
   ↪  mcp
   ```

3. Refaça o registro: `claude mcp remove algebraking -s user` e depois `./install.sh`.
4. Reinicie a sessão do Claude Code.

Timeouts. `Error, (in series) computation timed out after 2 s`: o motor foi reiniciado e a sessão continua. Aumente com `ALGEBRAKING_TIMEOUT=300`; se for um cálculo simbólico pesado, considere simplificá-lo à mão ou dividi-lo.

Cache sem permissão. `Error, cannot create .../.cache/algebraking: Permission denied`: o motor é gravado em `$XDG_CACHE_HOME/algebraking` (ou `~/.cache/algebraking`). Defina `XDG_CACHE_HOME` para um diretório gravável.

Depuração. `ALGEBRAKING_DEBUG=1 algebraking -e '...'` mostra o `stderr` do motor (`[engine] ...`) e os tracebacks Python de erros internos. Para testar o motor isolado:

```
echo '{"id":1,"op":"ping"}' | python3 engine/engine.py
```

Erro de pip "externally managed environment". O `install.sh` já tenta de novo com `--break-system-packages` (só no diretório do usuário). Se preferir, use um venv e aponte `ALGEBRAKING_PYTHON` para o Python dele, ou `sudo apt install python3-sympy python3-mpmath`.

---

9 Validação

O AlgebraKing reproduziu **257/257** verificações de a bateria de referência, vinda de um projeto de pesquisa (todas as 257 conferências deram o mesmo veredito).

Suítes de teste do repositório (executadas para escrever este guia):

```
cargo test --release                        # 39 testes: 26 unitários + 7 de
↪  CLI/golden + 6 de MCP, todos passando
python3 -m unittest engine/test_engine.py  # Ran 287 tests ... OK
```

Os testes "golden" (`tests/golden/*.mpl` e `*.out`) comparam a saída do binário com a saída esperada; os de MCP exercitam o handshake, as 8 ferramentas, o timeout e a reinicialização do motor.